

FACULDADE DE TECNOLOGIA - FATEC

Curso: [CURSO/SEMESTRE - INSERIR]

Disciplina: Projeto Atribuído - Integração Front-End / Back-End / Banco de Dados

BAB-STEAM-EPIC: Evolução do Projeto SteamTwo com Integração PostgreSQL e Docker Compose

Autor: Pedro Emanuel da Silva de Oliveira

RA: _____ [INSERIR RA]

Orientador: [INSERIR NOME DO PROFESSOR]

Cidade — 2026

BAB-STEAM-EPIC

Evolução do Projeto SteamTwo com Integração PostgreSQL e Docker
Compose

Pedro Emanuel da Silva de Oliveira

RA: _____

Trabalho apresentado como requisito da disciplina Projeto Atribuído

FOLHA DE ROSTO

BAB-STEAM-EPIC — Catálogo de jogos com dashboard de popularidade da Steam e Epic Games, evoluído a partir de <https://github.com/DavidSilvaProg/steamtwo>

- **Novo repositório:** <https://github.com/<SEU-USUARIO>/bab-steam-epic> [INSERIR LINK FINAL]
- **Pasta local:** /home/pedro/dev/Fatec/bab-steam-epic
- **Integrante:** Pedro Emanuel da Silva de Oliveira — RA: _____

Trabalho desenvolvido individualmente conforme rubrica (10 pontos).

SUMÁRIO

1. Introdução
 2. Referencial Teórico — SteamTwo
 3. Metodologia e Integração Back-End / Banco de Dados
 4. Melhorias Implementadas
 5. Instruções de Execução
 6. Testes e Resultados
 7. Evidências de Funcionamento (Prints)
 8. Conclusão
 9. Referências
-

1. INTRODUÇÃO

O projeto BAB-STEAM-EPIC tem como objetivo evoluir o projeto-base SteamTwo, consolidando a integração entre front-end (React/Vite), back-end (Node.js/Express) e banco de dados (PostgreSQL 17), conforme exigências da rubrica:

- 3,0 pts — back-end funcionando e integrado ao PostgreSQL
- 3,0 pts — melhorias relevantes
- 2,0 pts — integração, testes e qualidade técnica
- 2,0 pts — PDF completo conforme ABNT

O projeto foi desenvolvido solo, com foco prioritário na integração via Docker Compose.

2. REFERENCIAL — STEAMTWO

Funcionalidades originais:

- Dashboard com mais jogos agora, média da última semana, popularidade histórica e recorde monitorado
- Catálogo pesquisável e filtrável por loja e gênero
- Ranking combinado transparente: cada posição normalizada por $100 \times (N - \text{posição} + 1) / N$, índice combinado é média das fontes
- Coleta Steam, Epic Games e IGDB com snapshots imutáveis
- Fallback visual quando PostgreSQL não configurado

Stack: React 19 + Vite 6 (front em src/), Express 5 + pg 8 (back em server/), node-pg-migrate (migrations/001_create_steamtwo_domain.js), vitest + supertest.

3. INTEGRAÇÃO BACK-END COM BANCO DE DADOS

3.1 Arquitetura

```
[Browser] -> [Vite/React em src/ | dist/client]
           -> [Express server/index.js:14 em :3001]
           -> [createPool(DATABASE_URL) server/db/pool.js:6]
           -> [PostgreSQL 17 - postgres:17-alpine]
           -> [Tabelas: games, genres, game_genres, store_listings,
sync_runs, ranking_snapshots, ranking_entries, game_rankings]
```

- server/config.js:6 lê DATABASE_URL do .env
- server/index.js:14-18 cria pool e repository; se DATABASE_URL ausente, healthCheck retorna {status:"not-configured", mode:"demo"} (fallback)
- Com DB, healthCheck executa SELECT 1 e /api/health retorna {"status":"ok","database":{"status":"connected"}} (validado em curl http://127.0.0.1:3001/api/health)

3.2 Migrações

- migrations/001_create_steamtwo_domain.js cria 9 tabelas + 4 enums (store_name, sync_state, snapshot_status, ranking_period) + função steamtwo_prevent_snapshot_mutation() (snapshots imutáveis)

- Comandos: `npm run db:migrate (up)` e `npm run db:rollback (down)`
— testado: `MIGRATION 001_create_steamtwo_domain (UP)` com `CREATE TABLE` e `Migrations complete!`
- Evidência: `SELECT name, run_on FROM pgmigrations;` retorna `001_create_steamtwo_domain | 2026-08-31`, e \dt lista 9 relações

3.3 Docker Compose

Antes (original): apenas postgres em `docker-compose.yml:1` com `5432:5432`

Depois (BAB-STEAM-EPIC):

`services:`

```
postgres: # postgres:17-alpine, 5433:5432 (host 5433 para
          evitar conflito com postgres-db em 5432), healthcheck
          pg_isready, volume steamtwo_pgdata
api: # build Dockerfile (node:20-alpine, npm ci + npm run
      build), 3001:3001, DATABASE_URL=postgres://
      steamtwo:steamtwo@postgres:5432/steamtwo (interno),
      depends_on postgres healthy, healthcheck wget /api/health
migrate: # mesmo build, command npm run db:migrate, depende de
          postgres healthy
```

- Dockerfile multi-stage: `FROM node:20-alpine, npm ci, COPY .,`
`RUN npm run build` (gera `dist/client/index.html`, `dist/server/index.js`), `EXPOSE 3001`, `HEALTHCHECK wget /api/health`, `CMD ["node", "server/index.js"]`
- Host `DATABASE_URL=postgres://steamtwo:steamtwo@localhost:5433/steamtwo` (externo), container `DATABASE_URL=postgres://steamtwo:steamtwo@postgres:5432/steamtwo` (interno)
- Validação: `docker compose ps` mostra `bab-steam-epic-postgres-1 healthy` e `bab-steam-epic-api-1 healthy`, `docker logs bab-steam-epic-api-1` mostra `SteamTwo API disponível na porta 3001`

4. MELHORIAS IMPLEMENTADAS

Esta seção deve ser expandida conforme melhorias autorais forem adicionadas. Estrutura sugerida para cada melhoria:

#	Melhoria	Arquivo:linha	Descrição autoral	Evidência
1			Evoluiu compose de 1 para 3	

#	Melhoria	Arquivo:linha	Descrição autoral	Evidência
	Docker Compose completo	docker-compose.yml:1, Dockerfile:1	serviços, permitindo docker compose up --build reproduzível, sem dependência de Node local	docker compose ps healthy
2	Mapeamento de porta 5433	docker-compose.yml:9, .env:1	Evita conflito com postgres-db (5432) já em uso, documentado	docker ps 0.0.0.0:5433->5432
3	[INSERIR] Filtros/busca avançada	src/...	Busca com debounce, filtro por popularidade, ordenação	Print catálogo
4	[INSERIR] Dashboard	src/...	Gráfico 7 dias, card recorde	Print dashboard
5	[INSERIR] Validação zod + tratamento erros	server/routes/...	Middleware padronizado	Teste api 200/400

Atualmente entregues: itens 1 e 2 (integração). Itens 3-5 são próximos passos para completar 3,0 pts de melhorias.

5. INSTRUÇÕES PARA EXECUTAR O PROJETO

Requisitos: Docker 29+, Node 20+ (via fnm), Git

1. Clonar (ou usar pasta local)

```
git clone https://github.com/<SEU-USUARIO>/bab-steam-epic.git
cd bab-steam-epic
```

2. Instalar

```
fnm exec --using=lts-latest npm install
```

3. Configurar env (não commitar .env)

```
cp .env.example .env
```

```
# Editar se necessário: DATABASE_URL=postgres://
steamtwo:steamtwo@localhost:5433/steamtwo

# 4. Subir banco + api via Docker Compose (recomendado)
docker compose up -d --build
# ou apenas postgres para dev local:
# docker compose up -d postgres

# 5. Migrações
fnm exec --using=lts-latest npm run db:migrate
# Reverter se necessário: fnm exec --using=lts-latest npm run
db:rollback

# 6. Validar integração
curl http://127.0.0.1:3001/api/health
# Esperado: {"status":"ok","database":{"status":"connected"}}

# 7. Dev local (sem Docker api)
fnm exec --using=lts-latest npm run dev:api # :3001
fnm exec --using=lts-latest npm run dev # :5173

# 8. Produção via Docker (api serve front)
# Após build, acessar http://127.0.0.1:3001/

# 9. Testes
fnm exec --using=lts-latest npm test
fnm exec --using=lts-latest npm run build
fnm exec --using=lts-latest npm run test:sites

Frontend: http://127.0.0.1:3001/ (Docker) ou http://127.0.0.1:5173/
(Vite dev)
API: http://127.0.0.1:3001/api/health e http://127.0.0.1:3001/api/
games

Para IGDB: preencher TWITCH_CLIENT_ID e TWITCH_CLIENT_SECRET no .env
e executar npm run sync:catalog, sync:rankings, sync:popularity.
```

6. TESTES REALIZADOS E RESULTADOS

Teste	Comando	Resultado esperado	Resultado observado
Migração UP	npm run db:migrate		

Teste	Comando	Resultado esperado	Resultado observado
Migração DOWN/UP	db:rollback + db:migrate	Migrations complete! + 9 tabelas	OK — CREATE TABLE games... e Migrations complete!
		Reversível	OK — DROP TABLE e recriação
	psql -c "SELECT name FROM pgmigrations"	1 linha	OK — 001_create_steamtwo_domain
pgmigrations			OK — games, genres, game_genres, store_listings, sync_runs, ranking_snapshots, ranking_entries, game_rankings, pgmigrations
	psql -c "\dt"	9 tabelas	
/api/health	curl :3001/ api/health	connected	OK — {"status":"ok","database": {"status":"connected"}}
/api/games	curl :3001/ api/games	200 + pagination	OK — {"items": [], "pagination": {"page":1, "limit":12}} (vazio sem seed)
/(front)	curl :3001/	HTML	OK — <!doctype html><title>SteamTwo
npm test	npm test	23 passed	OK — 5 passed, 23 passed
npm build	npm run build	dist/ client/ index.html	OK — vite v6.4.2 built in 2.7s
test:sites	npm run test:sites	4 passed	OK — pass 4, fail 0
Docker health	docker compose ps	healthy	OK — postgres healthy, api health: starting -> healthy
Docker logs	docker logs api	porta 3001	OK — SteamTwo API disponível na porta 3001

Cobertura vitest: tests/api/routes.test.js (4), tests/domain/ranking.test.js (6), tests/integrations etc.

7. EVIDÊNCIAS DE FUNCIONAMENTO

Inserir aqui capturas de tela (substituir placeholders por prints reais):

1. `docker compose ps` — postgres 5433 e api 3001 healthy
2. `docker logs bab-steam-epic-api-1` — API na porta 3001
3. `curl /api/health` — JSON connected
4. `psql \dt e pgmigrations`
5. `npm test` — 23 passed
6. `npm run build` — dist/client + dist/server
7. Frontend em `http://127.0.0.1:3001/` — dashboard e catálogo
8. Dashboard com rankings (quando houver seed)

Para gerar evidências via terminal:

```
docker compose ps > docs/evidencia-ps.txt
curl -s http://127.0.0.1:3001/api/health | jq . > docs/evidencia-health.json
docker exec bab-steam-epic-postgres-1 psql -U steamtwo -d steamtwo -c "\dt" > docs/evidencia-dt.txt
```

8. CONCLUSÃO

A integração foi comprovada: Docker Compose orquestra Postgres 17 + API Node, migrações executam e `api/health` retorna connected. O sistema serve front e API na mesma porta em produção, mantendo compatibilidade com dev local (5173 + 3001). Próximos passos são expandir melhorias autorais (filtros, dashboard, validações) para completar 3,0 pts restantes.

9. REFERÊNCIAS

- DavidSilvaProg/steamtwo — <https://github.com/DavidSilvaProg/steamtwo>
- Documentação PostgreSQL 17, Docker, Node.js 20, Express 5, Vite 6
- ABNT NBR 14724:2011 — Trabalhos acadêmicos

ANEXO: Link do repositório: <https://github.com/<SEU-USUARIO>/bab-steam-epic>

Autor: Pedro Emanuel da Silva de Oliveira — RA: _____ (inserir)

Data: Agosto/2026